

CS771 - Homework 4 Writeup

Keegan Johnson, Forrest Peterson, Elice Priyadarshini, Jeffrey Weisinger

4.1) UNet

Understanding UNet (1pt)

- 1.) **Our implementation normalizes the input to the U-Net to be in the range of $[-1, 1]$ (see `./libs/datasets.py`). Can you explain why this is needed?**

The input to UNet needs to be normalized to the $[-1, 1]$ range to match the noise which exists in a similar range. Specifically, the DDPM paper (<https://arxiv.org/pdf/2006.11239>)

says the reverse process ultimately converges to pure noise: $p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$ (Ho. et al, page 2). Therefore, because we know the points at timestep T are centered around 0 with a standard deviation of 1, the image points (input to UNet) should similarly fall within a similar range in order for the added noise across T steps to have an appropriate impact (not too little, not too large). $[-1, 1]$ is an appropriate such range for the input and is thus chosen for UNet.

- 2.) **How does the U-Net implementation handle the input time t and condition c (image labels)? Please describe the operations that inject the t and c into feature maps.**

Time is used by first embedding t using a Sequential block with Sinusoidal function, followed by a Linear Layer, SILU activation, then another Linear Layer. Then, it is injected into the `mid_res_block`. before the attention operation (`mid_res_block1`), and in the `mid_res_block` after the attention operation (`mid_res_block2`). T

The image labels are used by embedding the class type c using a Sequential block with a Linear function, followed by a SILU activation, then again another Linear layer. Then, they are injected into the attention operation, which comes in between the `mid_res_block1` and `mid_res_block2`.

The idea is that we can use a residual network to learn the time input into the UNet, which will allow the model to predict noise with greater accuracy, as higher timesteps should have more noise. Similarly, we want the attention to learn the labels, so we can ensure the image is tuned to the desired class type.

Completing the implementation of UNet (1 pt)

We are asked to finish the implementation of UNet by completing the decoder portion of the forward function. This was accomplished by looping through each block in the decoder. For each block, we first add skip connections by concatenating the corresponding encoder output features with our current decoder features (x), which allows the decoder to access higher resolution details that were captured during downsampling. We then pass these concatenated

features and a time embedding (t) through a residual block, which helps the network learn what to do at different timesteps. Next we conditionally apply the transformer module (if it exists) on the features and a label embedding (c) in order to help the network attend to different parts of the image. Finally, we conditionally apply upsampling (if the upsampling module exists) to increase the spatial resolution, gradually restoring it back to the original image resolution.

4.2) DDPM and Latent DDPM

All math equation screenshots are taken from DDPM Paper (<https://arxiv.org/pdf/2006.11239>)

Forward diffusion process (1 pt)

In the implementation, we added noise to the original image across a span of T steps. To do this, we fill in q_sample which adds noise for just a single timestep.

We'd like to implement this function that adds noise for any single timestep,

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) := \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I})$$

However, for training we'd like to be able to train the model on all accumulated noise for a specific timestep without repeatedly applying the above formula t times to get there.

Therefore, we implement this alpha version which allows us to get the accumulated noise for timestep t, through just this one equation.

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I})$$

We repeat this for all x_start.shape[0] number of images (aka, regular images with no noise) in the input tensor.

Reverse denoising process (2 pts)

For this section, we'd like to implement a function that allows us to remove noise from the original image across a span of T steps.

To do this, we fill in p_sample which removes noise for a single timestep.

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}, \text{ where } \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

We first do this by calculating mu, which is the first part of this equation:

$$\mu_{\theta}(\mathbf{x}_t, t) = \tilde{\mu}_t \left(\mathbf{x}_t, \frac{1}{\sqrt{\bar{\alpha}_t}} (\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_{\theta}(\mathbf{x}_t, t)) \right) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right)$$

Further, we compute z as specified, understanding that $N(0, I)$ as presented in the paper is mean, covariance. We convert this covariance to standard deviation and pass into a `torch.normal` function (given that t_index is ≥ 1 , as specified in the algorithm)

Similarly to `q_sample`, we compute this in a batch. We are able to compute the equation for all `x.shape[0]` number of images (which are images with varying amounts of noise), and we return the slightly denoised image at timestep $t-1$.

In the `generate` function, we simply calculate `p_sample` T times (decreasing) to start with a pure noise image and create an image that resembles the original image distribution. Again, this `generate` function computes this process using a batch.

Simplified DDPM loss (1 pt)

For this loss, we also use the loss equation defined in the paper through the gradient descent function

Take gradient descent step on

$$\nabla_{\theta} \left\| \epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2$$

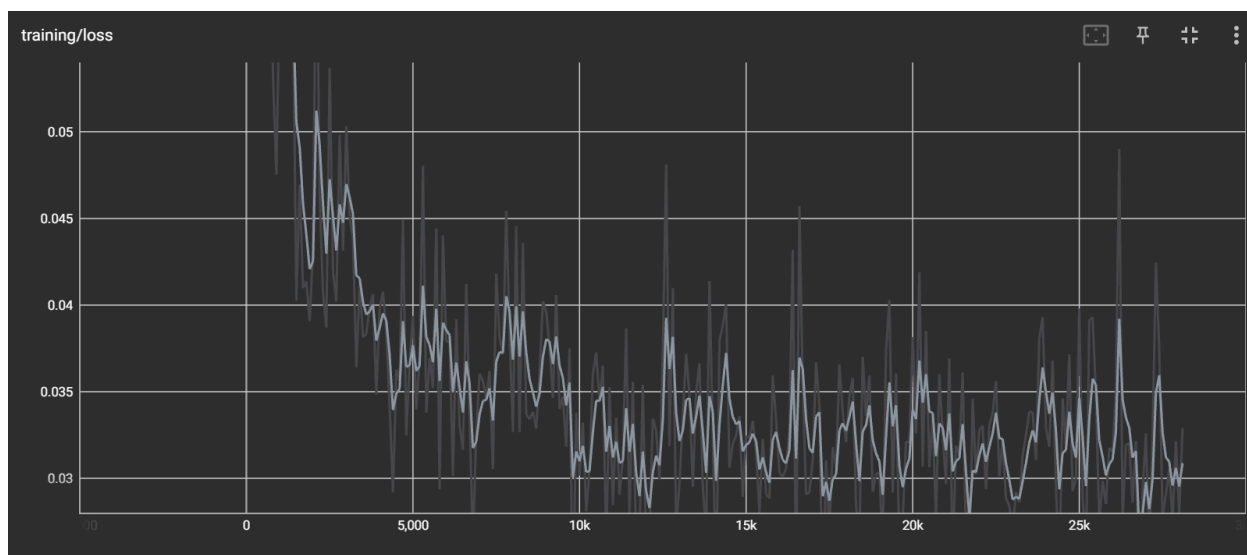
ϵ just represents random noise, but ϵ_{θ} represents the UNet model which is responsible for predicting noise. This makes sense, because we want ϵ_{θ} to be as close to ϵ as possible, which results in the smallest error. Then, we just define a mean squared error loss on the difference of these two errors.

Again, we are passed a tensor of x values, where each of the $x[i]$ values are a different image for which to predict ϵ . As before, we batch this process using `torch` functions.

Experimenting with DDPM on the MNIST dataset (1 pt)

We ran DDPM training for 30 epochs on the MNIST dataset.

Tensorboard Loss Logs:



Here are samples generated after the last epoch:



To analyze the final results, it can be helpful to see results from after epoch 9, and after epoch 19:

Post-Epoch 9 Results:



Post Epoch 19 Results:



Looking at these results, we can see that they somewhat resemble digits of human writing, but the final results are much crisper and easier to understand compared to results from epoch 9 and 19.

That being said, the final results seem to be quite high quality, and might be indistinguishable from normal human written digits to the untrained eye.

Further, the loss values seem to be slowly decreasing, and seem to follow an appropriate pattern.

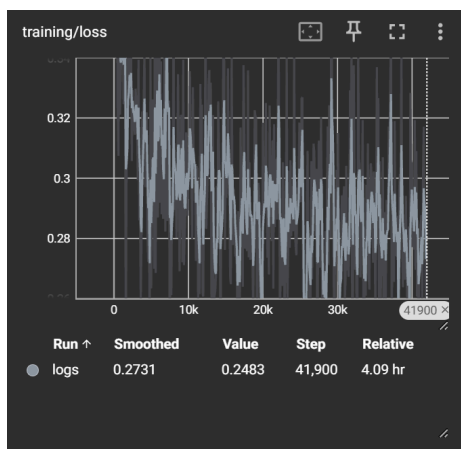
To compute this eval and training, we used the given mnist_ddpm.yaml file with no changes.

Latent Diffusion Models (LDMs) (1 pt)

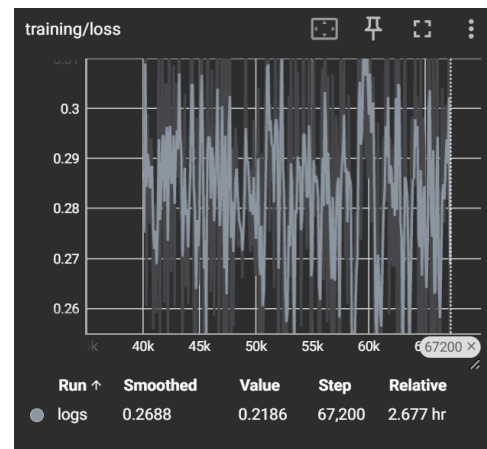
To enable image generation with higher resolution images, we implemented the latent diffusion model. This was accomplished through the use of a pretrained autoencoder. If the latent model was selected with the use_vae value, the input dimensions of the UNet and the output dimensions were changed to match the dimensions of the encoded state of the VAE. When training, rather than passing the full resolution images to the model for learning the denoising process, the images were first encoded using the encoding function of the autoencoder. Then the loss was computed based on the encoded images. Since the model learned in the latent space, to generate images the model generated the encoded representation of the images which were passed to the decoder. The decoder outputted the full resolution images which could then be post processed. This allowed the model to learn and generate high resolution images while using far less compute.

Experimenting with LDM on the AFHQ dataset (1 pt)

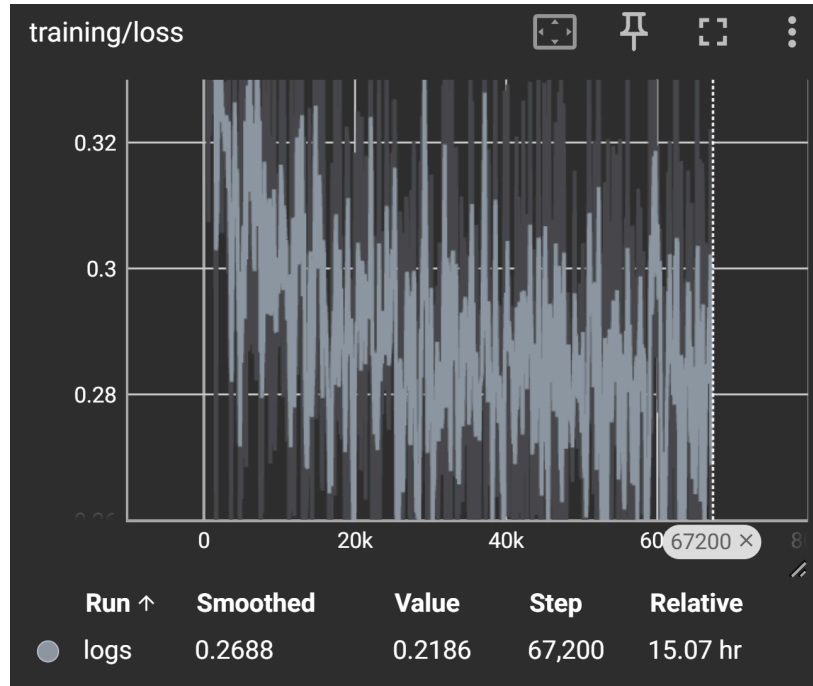
We ran the LDM on the AFHQ dataset for 300 epochs. The training logs for LDM with the AFHQ dataset are shown below. The training was interrupted part of the way through and resumed using a checkpoint, so the logs were divided and the training time for the combined logs is incorrect.



Training curve for first 180 epochs



Training curve for epochs 180-300



Full training curve for the LDM on the AFHQ dataset

As can be seen in the training curve, while the loss initially decreases drastically, it appears to plateau. The loss changes rather drastically and does not appear to improve much after about 150 epochs of training. This seems to indicate that the model has learned as much as it can with this configuration and dataset. The images appear to reflect this lack of improvement as they do not show drastic improvement after the first half of training.

Below are some of the images generated by the model after 300 epochs of training.



Sample images for the LDM model on the AFHQ dataset after 300 epochs

Upon first glance, these images are rather convincing and realistic. The cats are clearly cats, the dogs are clearly dogs, and the wild animals are clearly wolves, lions, cheetahs etc. When

examined closer, however, the defects become very obvious. Many of the eyes are warped and some of the edges of faces appear to be smeared. While not fantastic, the images are fairly impressive for being artificially generated.

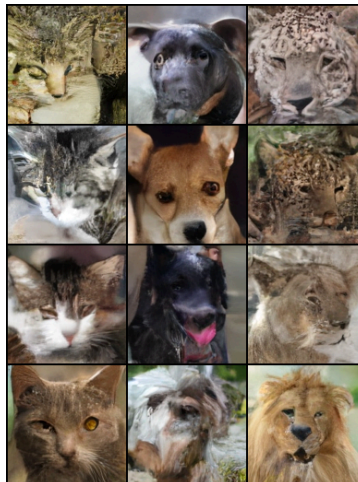
The progression of the model can be clearly seen by some of the sample images earlier in the model.



Sample images after 10 epochs



Sample images after 50 epochs



Sample images after 150 epochs

After 10 epochs the model generates fairly random images but the colors and shapes are already matching the images. After 50 epochs the model generates images resembling animals with a few very deformed images. By epoch 150 the results are fairly decent and not much different than the 300 epoch images, though there are a few extra issues. Ultimately, the images appear rather convincing.

Running the evaluation code on this model resulted in the following FID score:

70.87786189204763. This is rather good performance; however, the run time was rather long. In order to generate the 1500 images needed for evaluation, it took a little over 50 minutes. This is

likely due to the high computational cost of running the multiple step denoising needed to generate them.

FID: 70.87786189204763

Ultimately, while rather slow when generating images, the model performed well.

4.3) FM and Rectified Flow

Sampling process (1 pt)

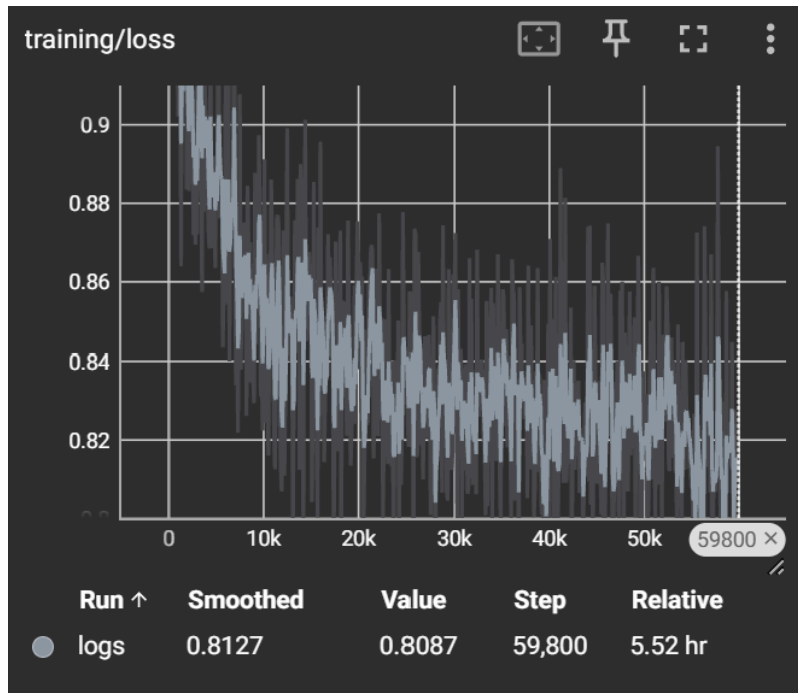
The Flow Matching sampling process is implemented in the FM class' generate() function. The first step is to generate uniformly spaced timesteps between 0 and 1, where each step advances time by dt ($1/\text{timesteps}$) along the flow trajectory. We start from pure Gaussian noise, which corresponds to $\tau=0$. Next, we loop through each timestep, iteratively following the flow from noise ($\tau=0$) to data ($\tau=1$). This is accomplished at each step by creating a batch of current time values for all images and then applying forward Euler integration: $x_{t+\Delta t} = x_t + \Delta t f_\theta(x_t, t)$, where f_θ is the velocity field predicted by the model at the current timestep. Finally, if we're using VAE for latent flow matching, we decode the latent representations back into image space.

Rectified Flow loss (1 pt)

We are also asked to implement the Flow Matching compute_loss() function, which is used to train the model to learn a velocity field that can transport noise into realistic images. The first step of this function is to conditionally (depending on the boolean use_vae variable) encode our initial image (x_{start}) into a smaller latent representation space. Next, we randomly select a time value τ from a uniform distribution over $[0, 1]$ for each image in the batch, where $\tau=0$ corresponds to pure Gaussian noise and $\tau=1$ corresponds to a clean image. We then construct the interpolated point x_τ by mixing the clean image and noise according to the formula: $x_\tau = \tau x_{\tau=1} + (1-\tau) x_{\tau=0}$, where the noise is either passed into the function or samples from a standard Gaussian distribution. This interpolated batch is then fed through the model to predict the velocity field $f_\theta(x_\tau, \tau)$. Finally we compute the MSE loss between the predicted velocity and the target velocity, which is given by $x_{\text{start}} - \text{noise}$ (representing the constant velocity needed to move from noise to data along a straight path).

Experimenting with FM on the AFHQ dataset (1 pt)

We trained the Fm model on the AFHQ dataset for 300 epochs. The logs and training curves can be seen below.



Training Curve for FM on AFHQ after 300 epochs

As can be seen in the graph, the error gradually drops throughout the training until near the end when it is relatively constant. There is an extra small drop at the very end though this might just be due to the instability of the training near the end.

Samples of the images after 300 epochs can be seen below.



Image samples of FM model on AFHQ dataset after 300 epochs

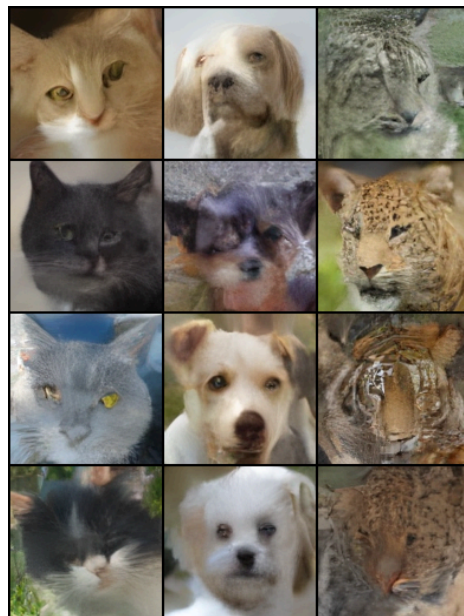
Just as with the LDM model, the images are rather convincing at first glance and each image matches its class type; however, when examining the images closer, they have several defects such as weird eyes and smeared features. When compared to the FDM results, these images definitely appear worse while not drastically different. The training images from during training do reflect the model learning throughout the process though.



Images after 20 epochs



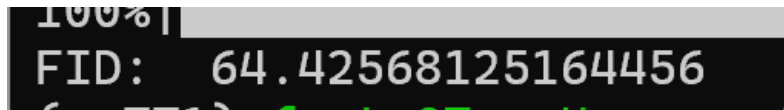
Images after 70 epochs



Images after 150 epochs

Just as with the FDM images, the model does not seem to improve too much after 150 epochs. The FM model does seem to learn faster in the initial stages. While the images are okay, the results are not incredible and still exhibit many defects.

After running the evaluation code on the FM model the result FID scores were: 64.42568125164456. This is slightly worse than the FID scores of the FDM model, but they are still rather comparable. While the performance was slightly worse, the FM model generated the 1500 images for evaluation substantially faster than the LDM model taking only a few minutes.



```
100% | ██████████  
FID: 64.42568125164456
```

Ultimately, the FM model performed well and generated decent images of animals in a faster time than the LDM model.

Contributions

Jeff and Elice: Developed DDPM Loss, DDPM Forward pass, DDPM Backward pass. Wrote "Understanding UNet Section," Trained on MNIST dataset. Added MNIST samples and training curves. Completed writeup for corresponding sections (4.1-4.2 up until and including MNIST experimentations).

Forrest and Keegan: Implemented the latent component of the LDM model, setup the VM platform for training the models, trained and evaluated the LDM model on the AFHQ dataset. Wrote the implementation for the FM generation code, and the FM compute loss code. Trained and evaluated the FM model on the AFHQ dataset. (All from LDM to 4.3).